

UNIT - 2

2 MARKS QUESTIONS

1. What are problem solving agents?

a problem-solving agent is an agent that operates by considering sequences of actions that lead to desirable states or outcomes.

2. What are the two types of example problems?

toy problem, Real-world problems

3. What is a toy problem? Give an example

A toy TOY PROBLEM problem is intended to illustrate or exercise various problem-solving methods. It can be given a concise, exact description. This means that it can be used easily by different researchers REAL-WORLD PROBLEM to compare the performance of algorithms.

Ex: 8-queens problem, 8-puzzle,

4. Give any two for real world example problems.

Automatic assembly sequencing ,Robot navigation, A VLSI layout, traveling salesperson problem

5. What is a fringe?

To represent the collection of nodes that have been generated but not yet FRINGE expanded-this collection is called the fringe. Each element of the fringe is a leaf node.

6. What are the problem formation and goal formation?

Problem formulation is the process of deciding what actions and states to consider, given a goal.

Goals help organize behavior GOAL FORMULATION by limiting the objectives that the agent is trying to achieve. Goal formulation, based on the current situation and the agent's performance measure, is the first step in problem solving.

7. What are the components of well-defined problem and solution?

problem

Initial State:

Successor Function:

State Space

Goal Test:

Path Cost Function

Solution:

Path:

Optimal Solution

8. Formulate the vacuum cleaner world problem

Initial State: (Dirty, Clean)

Actions: {Left, Right, Suck}

Goal Test: All squares are clean.

9. Formulate the vacuum cleaner world problem.

Initial State: (Dirty, Clean)

Actions: {Left, Right, Suck}

Goal Test: All squares are clean.

10. Formulate the 8-puzzle problem.

The standard formulation is as follows:

- States: A state description specifies the location of each of the eight tiles and the blank

§ in one of the nine squares.

- Initial state: Any state can be designated as the initial state. Note that any given goal

§ can be reached from exactly half of the possible initial states

- Successor function: This generates the legal states that result from trying the four
§ actions (blank moves Left, Right, Up, or Down).
- Goal test: This checks whether the state matches the goal configuration
- Path cost: Each step costs 1, so the path cost is the number of steps in the path.

11. Formulate the 8-Queen problem

- States: Any arrangement of 0 to 8 queens on the board is a state.
- Initial state: No queens on the board.
- Successor function: Add a queen to any empty square.
- Goal test: 8 queens are on the board, none attacked.
- States: Arrangements of n queens ($0 < n \leq 8$), one per column in the leftmost n
§ columns, with no queen attacking another are states.
- Successor function: Add a queen to any square in the leftmost empty column~~
such that
§ it is not attacked by any other queen

12. Define following

- a. Cell layout: In cell layout, the primitive components of the circuit are grouped into cells, each of which performs some recognized function.
- b. Channel layout.: Channel routing finds a specific route for each wire through the gaps between the cells. These search problems are extremely complex, but definitely worth solving

13. What are the operations on queue in tree search algorithms?

- MAKE-QUEUE(element, . . .) creates a queue with the given element(s).
- EMPTY?(queue) returns true only if there are no more elements in the queue.
- FIRST(queue) returns the first element of the queue.
- REMOVE-FIRST (queue) returns FIRST(queue) and removes it from the queue.

- INSERT(element, queue) inserts an element into the queue and returns the resulting queue. (We will see that different types of queues insert elements in different orders.)
- INSERT-ALL(elements, queue) inserts a set of elements into the queue and returns the resulting queue.

14. Define following

- a. Step Cost: A path cost function that assigns a numeric cost to each path.
- b. Path Cost: The step cost of taking action a to go from state x to state y is denoted by $c(x, a, y)$.

15. What are the four ways to measure an algorithm's performance?

COMPLETENESS

OPTIMALITY

TIME COMPLEXITY

SPACE COMPLEXITY

16. Distinguish between informed and uninformed search strategies.

Parameters	Informed Search	Uninformed Search
Known as	is also known as Heuristic Search.	It is also known as Blind Search.
Using Knowledge	uses knowledge for the searching process.	doesn't use knowledge for the searching process.

Performance	it finds a solution more quickly.	finds solution slow as compared to an informed search.
Completion	it may or may not be complete.	It is always complete.
Cost Factor	Cost is low.	Cost is high.

17.What are uninformed search strategies? Give an example

Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

Following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search
6. Bidirectional Search

18.What are informed search strategies? Give an example

informed search strategy--one that uses problem-specific knowledge beyond the definition of the problem itself-can find solutions more efficiently than an uninformed strategy.

Greedy best-first search, A* SEARCH,

19. Give the time and space complexity of Breadth First Search.

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d = depth of shallowest solution and b is a node at every state.

$$T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

20. What is the difference between Breadth First search and Uniform Cost

Breadth-First Search (BFS):

- BFS expands nodes level by level, exploring all nodes at the current level before moving to nodes at the next level.
- It does not consider the cost of the path from the start node to the current node. Instead, it focuses solely on the breadth of the search, ensuring that all nodes at the current level are explored before moving to deeper levels.
- BFS guarantees the shortest path to the goal if the edge costs are uniform (i.e., all edges have the same cost), as it explores nodes in increasing order of their depth from the start node.

Uniform Cost Search (UCS):

- UCS expands nodes based on their cumulative path cost from the start node. It always selects the node with the lowest cumulative cost for expansion.
- Unlike BFS, UCS considers the cost of the path from the start node to the current node. It prioritizes nodes with lower path costs, ensuring that it explores the cheapest paths first.
- UCS guarantees finding the optimal solution in terms of path cost, regardless of whether the edge costs are uniform or not, as it systematically explores paths in increasing order of cost.

Search?

21. Give the time and space complexity of Depth First Search.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.

22. Give the time and space complexity of Uniform Cost Search.

Time Complexity:

Let C^* is **Cost of the optimal solution**, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken $+1$, as we start from state 0 and end to C^*/ϵ .

Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

23. Give the time and space complexity of bidirectional search.

Time Complexity: Time complexity of bidirectional search using BFS is $O(b^d)$.

Space Complexity: Space complexity of bidirectional search is $O(b^d)$.

24. Give the time and space complexity of Depth limited search.

Time Complexity: Time complexity of DLS algorithm is $O(b^l)$.

Space Complexity: Space complexity of DLS algorithm is $O(b \times l)$.

25. Give the time and space complexity of Iterative deepening depth first

search.

Time Complexity: Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity: The space complexity of IDDFS will be $O(bd)$.

26. Give the time and space complexity of Greedy Best First Search.

The worst-case time and space complexity is $O(bm)$, where m is the maximum depth of the search space

27. Give the time and space complexity of A* search algorithm.

the algorithm can be $O(b^d)$, where b is the branching factor

The space complexity of standard A* is always $O(b^d)$,

28. What is bidirectional search?

Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

29. List any two types of problems while searching with incomplete

information.

1. Sensorless problems
2. Contingency problems
3. Exploration problems:

30. What are heuristic functions? Give example.

There is a whole family of BEST-FIRST-SEARCH algorithms with different evaluation HEURISTIC FUNCTION functions.' A key component of these algorithms is a heuristic function,

denoted $h(n)$: $h(n)$ = estimated cost of the cheapest path from node n to a goal node.

For example, in Romania, one might estimate the cost of the cheapest path from Arad to Bucharest via the straight-line distance from Arad to Bucharest.

31. When do you say a heuristic is admissible?

The heuristic is admissible because the optimal solution in the original problem is, by definition, also a solution in the relaxed problem and therefore must be at least as expensive as the optimal solution in the relaxed problem.

32. When do you say a heuristic is consistent?

Because the derived heuristic is an exact cost for the relaxed problem, it must obey the triangle inequality and is therefore consistent

.A heuristic $h(n)$ is consistent if, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' :

33. Write a brief note on Iterative Deepening A* algorithm.

The simplest way to reduce memory requirements for A* is to adapt the idea of iterative deepening to the heuristic search context, resulting in the iterative-deepening A* (IDA*) algorithm. IDA* is practical for many problems with unit step costs and avoids the substantial overhead associated with keeping a sorted queue of nodes.

34. List any two memory bound algorithms.

iterative-deepening A* (IDA*) algorithm

Recursive best-first search (RBFS)

MA* (memory-bounded A*)

35. What is Absolver?

A program called ABSOLVER can generate heuristics automatically from problem definitions, using the "relaxed problem" method and various other techniques

36. What are the key advantages of local search algorithms?

(1) they use very little memory-usually a constant amount;

(2) they can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

37.What is local search?

Local search algorithms operate using a single current state (rather than multiple paths) and generally move only to neighbors of that state. Typically, the paths followed by the search are not retained. Although local search algorithms are not systematic

38.List various steps of genetic algorithms.

Initial Population

Fitness Function

selection

Crossover

Mutation

39.What are flat local maximum and shoulder state.

plateau is an area of the state space landscape where the evaluation function is flat. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which it is possible to make progress.

40. What is local maximum and global maximum

a local maximum is a peak that is higher than each of its neighboring states, but lower than the global maximum

if elevation corresponds to an objective function, then the aim is to find the highest peak-a global GLOBALMAXIMUM maximum

41.List any two variants of hill climbing search.

Stochastic hill climbing, First-choice hill climbing

42.Find the two types of heuristic values for the 8 puzzle instance given

h_1 = the number of misplaced tiles

h_2 = city block distance or Manhattan distance

below:

Long Answer Questions (3.4.5.6 Marks)

1. Explain a problem solving agent with algorithm.

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action

inputs: percept, a percept

static: seq, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state ← UPDATE-STATE(state, percept)

if seq is empty then do

goal ← FORMULATE-GOAL(state)

problem ← FORMULATE-PROBLEM(state, goal)

seq ← SEARCH(problem)

action ← FIRST(seq)

seq ← REST(seq)

return action

A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over. Note that when it is executing the sequence it ignores its percepts: it assumes that the solution it has found will always work.

2. Write a note on well-defined problems and solutions.

A problem can be defined formally by four components:

1. INITIAL STATE: The initial state that the agent starts in. For example, the initial state for our agent in Romania might be described as In(Arad).
2. SUCCESSOR FUNCTION: A description of the possible actions available to the agent. The most common for SUCCESSOR FUNCTION mutation3 uses a successor function. Given a particular state x , SUCCESSOR-FN(x) returns a

set of (action, successor) ordered pairs, where each action is one of the legal actions in state x and each successor is a state that can be reached from x by applying the action. F

STATE SPACE: Together, the initial state and successor function implicitly define the state space of the problem-the set of all states reachable from the initial state. The state space forms a graph in which the nodes are states and the arcs between nodes are actions

3. GOAL TEST: The goal test, which determines whether a given state is a goal state. Sometimes there is an explicit set of possible goal states, and the test simply checks whether the given state is one of them.
4. PATH COST: A path cost function that assigns a numeric cost to each path. The problem-solving agent chooses a cost function that reflects its own performance measure.

OPTIMAL SOLUTION: The preceding elements define a problem and can be gathered together into a single data structure that is given as input to a problem-solving algorithm. A solution to a problem is a path from the initial state to a goal state. Solution quality is measured by the path cost function, and an optimal solution has the lowest path cost among all solutions

3. Explain the 8-puzzle problem

The 8-puzzle, an instance of which is shown in Figure 3.4, consists of a 3 x 3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the

space. The object is to reach a specified goal state, such as the one shown on the right of the

figure. The standard formulation is as follows:

- States: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- Initial state: Any state can be designated as the initial state. Note that any given goal

can be reached from exactly half of the possible initial states\

- Successor function: This generates the legal states that result from trying the four

actions (blank moves Left, Right, Up, or Down).

- Goal test: This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- Path cost: Each step costs 1, so the path cost is the number of steps in the path

The 8-puzzle belongs to the family of sliding-block puzzles, which are often used as test problems for new search algorithms in AI. The 8-puzzle has $9!/2 = 181,440$ reachable states and is easily solved. (give one example for 8 puzzle)

4. Explain the 8 queen problem

The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other. (A queen attacks any piece in the same row, column or diagonal.) Figure 3.5 shows an attempted solution that fails: the queen in the rightmost column is attacked by the queen at the top left.

There are two main kinds of formulation. An incremental formulation involves operators that augment the state description, starting with an empty state; for the 8-queens problem, this means that each action adds a queen to the state. A complete-state formulation starts with all 8 queens on the board and moves them around. In either case, the path cost is of no interest because

only the final state counts. The first incremental formulation one might try is the following:

- States: Any arrangement of 0 to 8 queens on the board is a state.
- Initial state: No queens on the board.
- Successor function: Add a queen to any empty square.
- Goal test: 8 queens are on the board, none attacked

A better formulation would prohibit placing a queen in any square that is already attacked:

- States: Arrangements of n queens ($0 < n \leq 8$), one per column in the leftmost n columns, with no queen attacking another are states.
- Successor function: Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

This formulation reduces the 8-queens state space from 3×10^{14} to just 2,057, and solutions are easy to find

5. Explain BFS algorithm with example

Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d = depth of shallowest solution and b is a node at every state.

$$T(b) = 1 + b^2 + b^3 + \dots + b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

6. Explain Uniform cost searching algorithm with example

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs from the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Advantages:

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Example:

Completeness:

Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity:

Let C^* is **Cost of the optimal solution**, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken $+1$, as we start from state 0 and end to C^*/ϵ .

Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Optimal:

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

7. Explain DFS algorithm with example

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Note: Backtracking is an algorithm technique for finding all possible solutions using recursion.

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.

- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

8. Explain Depth limited search algorithm with example

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

Advantages:

Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Example:

Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $O(b^l)$.

Space Complexity: Space complexity of DLS algorithm is $O(b \times l)$.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $l > d$.

9. Explain Iterative deepening depth first search algorithm with example

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Advantages:

- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Example:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:

1'st Iteration-----> A

2'nd Iteration----> A, B, C

3'rd Iteration----->A, B, D, E, C, F, G

4'th Iteration----->A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete is if the branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity:

The space complexity of IDDFS will be $O(bd)$.

Optimal:

IDDFS algorithm is optimal if path cost is a non-decreasing function of the depth of the node.

10.Explain Bidirectional search algorithm with example

Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- **In bidirectional search, one should know the goal state in advance.**

Example:

In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.

Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is $O(b^d)$.

Space Complexity: Space complexity of bidirectional search is $O(b^d)$.

Optimal: Bidirectional search is Optimal.

11.What are toy example problems? Explain any one toy problem in detail.

Refer question 3 or 4(6marks)

A toy TOY PROBLEM problem is intended to illustrate or exercise various problem-solving methods. It can be given a concise, exact description. This means that it can be used easily by different researchers REAL-WORLD PROBLEM to compare the performance of algorithms.

Ex: 8-queens problem, 8-puzzle,

Refer question 3 or 4(6marks)

12.Explain the vacuum world problem.

13.Explain any two real world example problems.

2 real world examples are:

1. Traveling salesperson problem (TSP)

The traveling salesperson problem (TSP) is a touring problem in which each city must be visited exactly once. The aim is to find the shortest tour. The problem is known to be NP-hard, but an enormous amount of effort has been expended to improve the capabilities of TSP algorithms. In addition to planning trips for traveling salespersons, these algorithms have been used for tasks such as planning movements of automatic circuit-board drills and of stocking machines on shop floors.



Problem formulation

State: N cities

Initial state: Any source city

Action: Moving the neighboring city in the path

Transition model: reaching next city based on the action

Goal test: Checking for the optimal tour path (Hamiltonian path)

Path cost: Optimal tour cost

2. VLSI LAYOUT

A VLSI layout problem requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield. The layout problem comes after the logical design phase, and is usually split

into two parts: cell layout and channel routing. In cell layout, the primitive components of the circuit are grouped into cells, each of which performs some recognized function. Each cell

has a fixed footprint (size and shape) and requires a certain number of connections to each of the other cells. The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells. Channel routing finds a specific

route for each wire through the gaps between the cells.

Problem definition

positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield.



It comes with two parts:

1. Cell layout : Placing primitive components of the circuit are grouped into cells, each of which performs some **recognized function**. Each cell has a fixed **footprint (size and shape)** and requires a certain number of connections to each of the other cells. The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells.

2. Channel routing: finds a specific route for each wire through the gaps between the cells

14.Explain the various components of a node.

A node is a data structure with five components:

STATE a state in the state space to which the node corresponds,

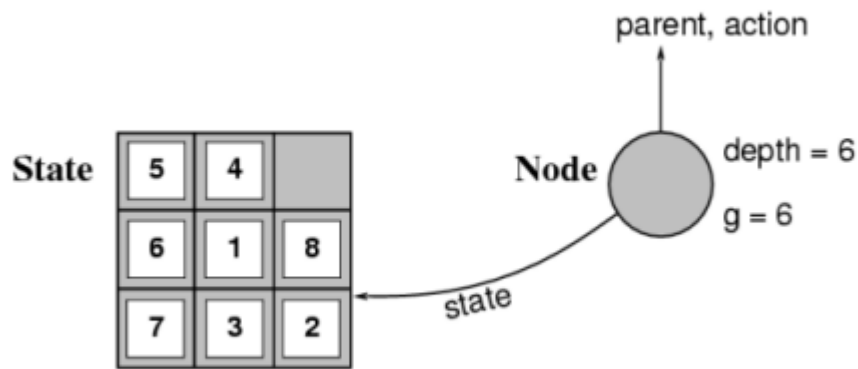
PARENT-NODE: the node in the search tree that generated this node;

ACTION: the action that was applied to the parent to generate the node,

PATH-COST $g(n)$, cost of the path from initial state to the node, as indicated by the parent pointers; and

DEPTH: the number of steps along the path from the initial state.

It is important to remember the distinction between nodes and states. A node is a book keeping data structure used to represent the search tree. A state corresponds to configuration of the



world.

15. Write and explain tree search algorithm.

Breadth-first search is a simple strategy in which the root node is expanded first, then all the **BREADTH-FIRST** successors of the root node are expanded next, then their successors, and so on. In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded. Breadth-first search can be implemented by calling **TREE-SEARCH** with an empty fringe that is a first-in-first-out (FIFO) queue, assuring that the nodes that are visited first will be expanded first. In other words, calling results in a breadth-first search. The FIFO queue puts all newly generated successors at the end of the queue, which means that shallow nodes are expanded before deeper nodes. Figure 3.10 shows the progress of the search on a simple binary tree. We will evaluate breadth-first search using the four criteria from the previous section. We can easily see that it is complete-if the shallowest goal node is at some finite depth d , breadth-first search will eventually find it after expanding all shallower nodes (provided the branching factor b is finite). The shallowest goal node is not necessarily the optimal one; technically, breadth-first search is optimal if the path cost is a nondecreasing function of the depth of the node. (For example, when all actions have the same cost.) So far, the news about breadth-first search has been good. To see why it is not

always the strategy of choice, we have to consider the amount of time and memory it takes to complete a search. To do this, we consider a hypothetical state space where every state has b successors. The root of the search tree generates b nodes at the first level, each of which generates b more nodes, for a total of b^2 at the second level. Each of these generates b more nodes, yielding b^3 nodes at the third level, and so on. Now suppose that the solution is at depth d . In the worst 74 Chapter 3. Solving Problems by Searching case, we would expand all but the last node at level d (since the goal itself is not expanded), generating $bdS1 - b$ nodes at level d f 1. Then the total number of nodes generated is

Every node that is generated must remain in memory, because it is either part of the fringe or is an ancestor of a fringe node. The space complexity is, therefore, the same as the time complexity (plus one node for the root). Those who do complexity analysis are worried (or excited, if they like a challenge) by exponential complexity bounds such as $O(b^{d+1})$. Figure 3.1 1 shows why. It lists the time and memory required for a breadth-first search with branching factor $b = 10$, for various values of the solution depth d . The table assumes that 10,000 nodes can be generated per second and that a node requires 1000 bytes of storage. Many search problems fit roughly within these assumptions (give or take a factor of 100) when run on a modern personal computer. There are two lessons to be learned from The second lesson is that the time requirements are still a major factor. If your problem has a solution at depth 12, then (given our assumptions) it will take 35 years for breadth-first search (or indeed any uninformed search) to find it. In general, exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

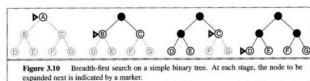


Figure 3.10 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

Depth	Nodes	Time	Memory
2	100	.11 seconds	1 megabyte
4	111,100	11 seconds	106 megabytes
6	10^6	19 minutes	10 gigabytes
8	10^8	31 hours	1 terabyte
10	10^{10}	129 days	101 terabytes
12	10^{12}	35 years	10 petabytes
14	10^{14}	3,523 years	1 exabyte

Figure 3.11 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$, 10,000 nodes/second, 1000 bytes/node.

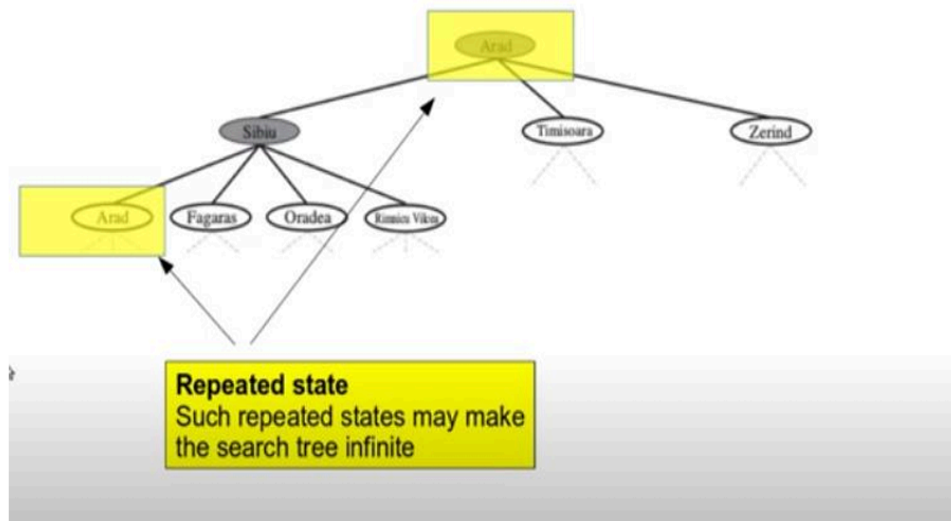
16.Explain Breadth First Search. Also discuss its performance.

Refer question5

17.. Explain how to avoid repeated states. Also write the General Graph Search

Algorithm.

Repeated states Failure to detect repeated states can turn a linear problem into an exponential one.



```

function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
  
```

If an algorithm remembers every state that it has visited, then it can be viewed as exploring the state-space graph directly. We can modify the general TREE-SEARCH algorithm to include a data structure called the closed list, which stores every expanded node. (The fringe of unexpanded nodes is sometimes called the open list.)

If an algorithm remembers every state that it has visited, then it can be viewed as exploring the state-space graph directly. We can modify the general TREE-SEARCH algorithm to include a data structure called the closed list, which stores every expanded node. (The fringe of unexpanded nodes is sometimes called the open list.)

If the current node matches a node on the closed list, it is discarded instead of being expanded. The new algorithm is called GRAPH-SEARCH (Figure 3.19). On problems with many repeated states, GRAPH-SEARCH is much more efficient than TREE-SEARCH. Its worst-case time and space requirements are proportional to the size of the state space. This may be much smaller than $O(bd)$.

Graph search

```

function GRAPH-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
    only if not in the frontier or explored set

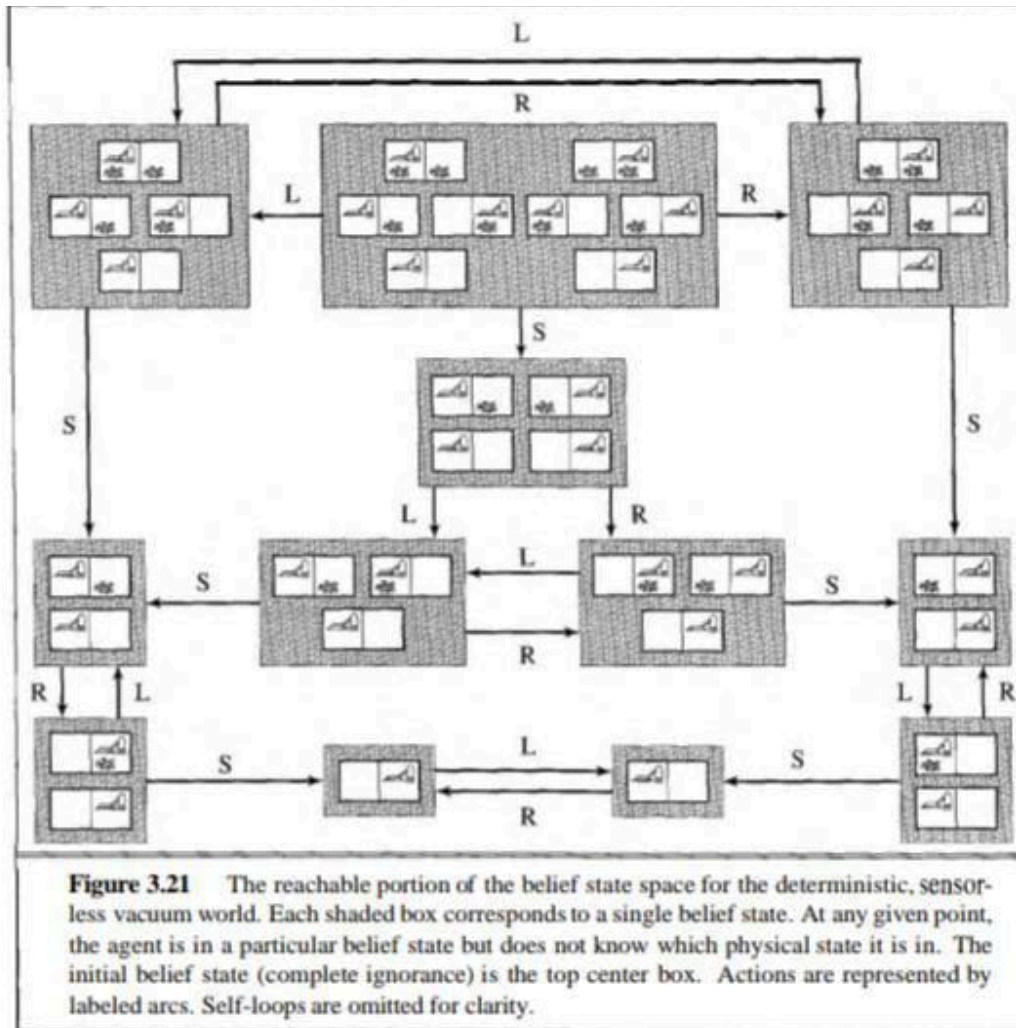
```

```

function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe ← INSERTALL(EXPAND(node, problem), fringe)

```

18. Write a note on conformant problem (Sensorless problems)



Sensorless problems

Suppose that the vacuum agent knows all the effects of its actions, but has no sensors. Then it knows only that its initial state is one of the set $\{1,2,3,4,5,6,7,8\}$. One might suppose that the agent's predicament is hopeless, but in fact it can do quite well. Because it knows what its actions do, it can, for example, calculate that the action Right will cause it to be in one of the states $\{2,4,6,8\}$, and the action sequence [Right,Suck] will always end up in one of the states $\{4,8\}$. Finally, the sequence [Right,Suck,Left,Suck] is guaranteed to reach the COERCION goal state 7 no matter what the start state. We say that the agent can coerce the world into

state 7, even when it doesn't know where it started. To summarise: when the world is not fully observable, the agent must reason about sets of states that it might get to, rather than BELIEF STATE single states. We call each such set of states a belief state, representing the agent's current

belief about the possible physical states it might be in. (In a fully observable environment, each belief state contains one physical state.)

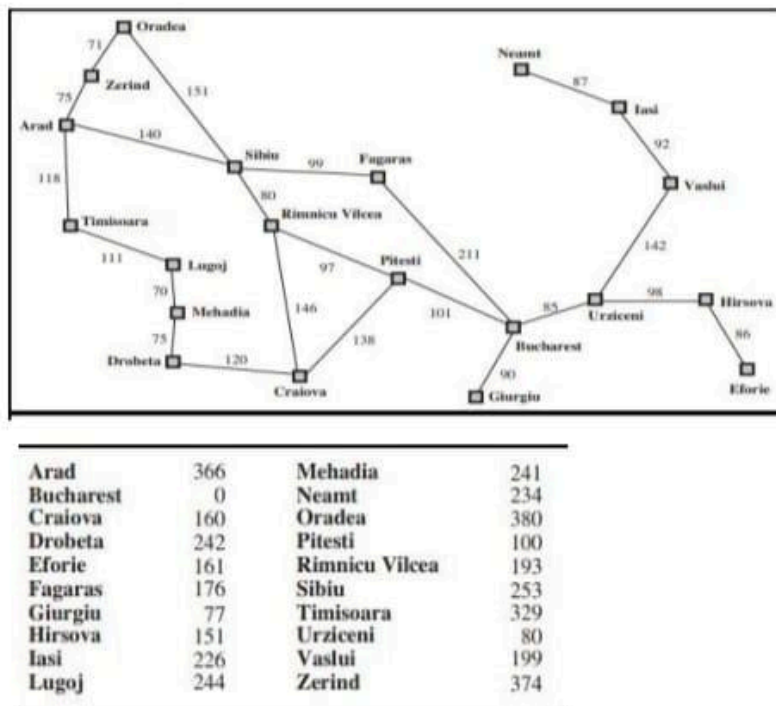
To solve sensorless problems, we search in the space of belief states rather than physical states. The initial state is a belief state, and each action maps from a belief state to another belief state. An action is applied to a belief state by unioning the results of applying the action to each physical state in the belief state. A path now connects several belief states, and a solution is now a path that leads to a belief state, all of whose members are goal states.

Figure 3.21 shows the reachable belief-state space for the deterministic, sensorless vacuum world. There are only 12 reachable belief states, but the entire belief state space contains every possible set of physical states, i.e., $2^8 = 256$ belief states. In general, if the physical state space has S states, the belief state space has 2^S belief states.

Our discussion of sensorless problems so far has assumed deterministic actions, but the analysis is essentially unchanged if the environment is nondeterministic—that is, if actions may have several possible outcomes. The reason is that, in the absence of sensors, the agent has no way to tell which outcome actually occurred, so the various possible outcomes are just additional physical states in the successor belief state. For example, suppose the environment obeys Murphy's Law: the so-called Suck action sometimes deposits dirt on the carpet but only if there is no dirt there already.⁶ Then, if Suck is applied in physical state 4 (see Figure 3.20), there are two possible outcomes: states 2 and 4. Applied to the initial belief state, $\{1,2,3,4,5,6,7,8\}$, Suck now leads to the belief state that is the union of the outcome sets for the eight physical states. Calculating this, we find that the new belief state is

{1,2,3,4,5,6,7,8). So, for a sensorless agent in the Murphy's Law world, the Suck action leaves the belief state unchanged! In fact, the problem is unsolvable. (See Exercise 3.18.) Intuitively, the reason is that the agent cannot tell whether the current square is dirty and hence cannot tell whether the Suck action will clean it up or create more dirt.

19. Explain Greedy Best First Search. Consider the map of Romania and the hSLD value table given below. Use greedy best-first search to find a path from Arad to Bucharest.



Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function: $f(n) = h(n)$. Let us see how this works for route-finding problems in Romania, using the straight-line distance heuristic, which we will call hSLD. If the goal is Bucharest, we will need to know the straight-line distances to Bucharest, which are shown in Figure 4.1. For example, $hSLD(\text{In}(\text{Arad})) = 366$. Notice that the values of hSLD cannot be computed from the problem

description itself. Moreover, it takes a certain amount of experience to know that hSLD is correlated with actual road distances and is, therefore, a useful heuristic.

shows the progress of a greedy best-first search using hSLD to find a path from Arad to Bucharest. The first node to be expanded from Arad will be Sibiu, because it is closer to Bucharest than either Zerind or Timisoara. The next node to be expanded will be Fagaras, because it is closest. Fagaras in turn generates Bucharest, which is the goal. For this particular problem, greedy best-first search using hSLD finds a solution without ever expanding a node that is not on the solution path; hence, its search cost is minimal. It is not optimal, however: the path via Sibiu and Fagaras to Bucharest is 32 kilometers longer than the path through Rimnicu Vilcea and Pitesti. This shows why the algorithm is called "greedy2"-at each step it tries to get as close to the goal as it can. Minimizing $h(n)$ is susceptible to false starts. Consider the problem of getting from Iasi to Fagaras. The heuristic suggests that Neamt be expanded first, because it is closest to Fagaras, but it is a dead end. The solution is to go first to Vaslui-a step that is actually farther from the goal according to the heuristic-and then to continue to Urziceni, Bucharest, and Fagaras. In this case, then, the heuristic causes unnecessary nodes to be expanded. Furthermore, if we are not careful to detect repeated states, the solution will never be found-the search will oscillate between Neamt and Iasi. Greedy best-first search resembles depth-first search in the way it prefers to follow a single path all the way to the goal, but will back up when it hits a dead end. It suffers from the same defects as depth-first search-it is not optimal, and it is incomplete (because it can start down an infinite path and never return to try other possibilities). The worst-case time and space complexity is $O(bm)$, where m is the maximum depth of the search space. With a good heuristic function, however, the complexity can be reduced substantially. The amount of the reduction depends on the particular problem and on the quality of the heuristic.

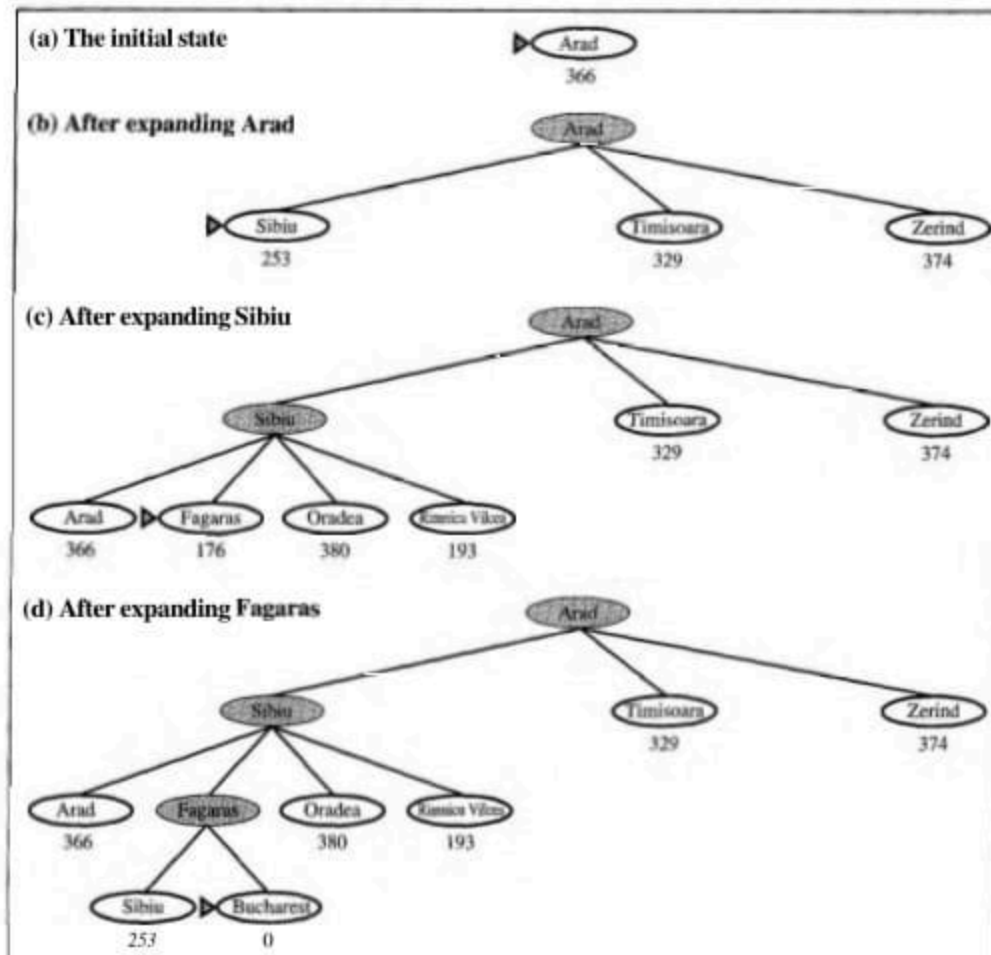


Figure 4.2 Stages in a greedy best-first search for Bucharest using the straight-line distance heuristic h_{SLD} . Nodes are labeled with their h-values.

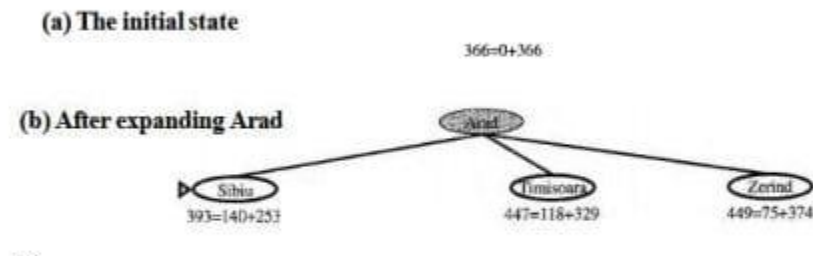
20. Explain A* algorithm. Consider the map of Romania and the hSLD value table given. Use A* algorithm to find a path from Arad to Bucharest.

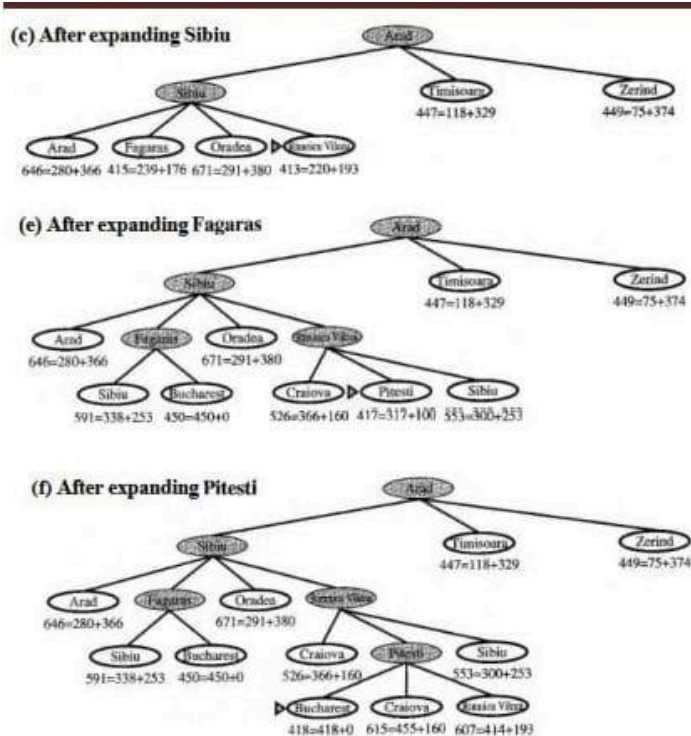
A* search is a popular algorithm used to find the shortest path between two points. It works by evaluating nodes based on two costs: $g(n)$, the cost to reach the node, and $h(n)$, the estimated cost to get from the node to the goal. The algorithm chooses the node with the lowest

combined cost, $g(n) + h(n)$, to explore next. This strategy is not only reasonable but also guarantees the shortest path if the heuristic function $h(n)$ meets certain conditions.

The optimality of A* search relies on the use of an admissible heuristic, which never overestimates the cost to reach the goal. Admissible heuristics are optimistic, thinking the cost is less than it actually is. Since $g(n)$ is the exact cost to reach a node, the combined cost $f(n) = g(n) + h(n)$ never overestimates the true cost of a solution. A simple example of an admissible heuristic is the straight-line distance between two points, which is always an underestimate.

A* search is used with the TREE-SEARCH algorithm to find the shortest path. In this example, the algorithm is used to find the shortest path to Bucharest. The values of $g(n)$ are computed from the step costs, and the values of $h(n)$ are given as the straight-line distance. The algorithm chooses the node with the lowest combined cost to explore next. In this case, Bucharest appears on the fringe but is not selected because its cost is higher than that of Pitesti. The algorithm will not settle for a solution that costs more than the potential solution through Pitesti.





21. Explain Recursive Best First Search algorithm. Consider the map of Romania and the hSLD value table given. Use RFBS* algorithm to find a path from Arad to Bucharest.

Recursive Best-First Search (RBFS) Algorithm

RBFS is a search algorithm that uses a best-first strategy but limits memory usage by using recursion and backtracking with minimal state information. It keeps track of the best (lowest cost) option found so far and recursively explores the best paths.

Steps to Find a Path from Arad to Bucharest

Initialize the search:

1. Start from the initial node (Arad).
2. Use the heuristic hSLD values given for each city to guide the search.

Expand nodes:

1. At each node, expand the available successors.
2. Calculate the estimated cost $f(n)=g(n)+h(n)$, where:

$g(n)$ is the cost to reach node n from the start.

$h(n)$ is the heuristic value (straight-line distance) from n to the goal (Bucharest).

Select the best node:

1. Choose the node with the lowest estimated cost $f(n)$ for expansion.
2. If the current path exceeds the best alternative cost, backtrack.

Recur and prune:

1. Recursively apply the process, keeping track of the best path cost found so far.
2. Prune paths that exceed the best alternative cost.

Terminate:

1. The process terminates when the goal (Bucharest) is reached or no better paths are available.

Step-by-Step Execution

1. Start at Arad

$$f(\text{Arad}) = g(\text{Arad}) + h(\text{Arad}) = 0 + 366 = 366$$

2. Expand Arad

Successors of Arad:

Sibiu: $g=140$, $h=253$, $f=140+253=393$.

Timisoara: $g=118$, $h=329$, $f=118+329=447$.

Zerind: $g=75$, $h=374$, $f=75+374=449$.

Choose Sibiu (lowest f -value).

3. Expand Sibiu

Successors of Sibiu:

Arad (already visited, skip).

Fagaras: $g=140+99=239$, $h=176$, $f=239+176=415$.

Rimnicu Vilcea: $g=140+80=220$, $h=193$, $f=220+193=413$.

Oradea: $g=140+151=291$, $h=380$, $f=291+380=671$.

Choose Rimnicu Vilcea (lowest f-value).

4. Expand Rimnicu Vilcea

Successors of Rimnicu Vilcea:

Sibiu (already visited, skip).

Pitesti: $g=220+97=317$, $h=100$, $f=317+100=417$.

Craiova: $g=220+138=358$, $h=160$, $f=358+160=518$.

Choose Pitesti (lowest f-value).

5. Expand Pitesti

Successors of Pitesti:

Rimnicu Vilcea (already visited, skip).

Bucharest: $g=317+101=418$, $h=0$, $f=418+0=418$.

Choose Bucharest.

Path Found:

- **Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest.**
- Total cost: 418.

22. Write a note on SMA* algorithm.

23. Write a note on IDA* algorithm.

IDA* (Iterative Deepening A*) is a graph traversal and pathfinding algorithm that combines the benefits of both A* (A-star) and iterative deepening depth-first search (IDDFS). It is primarily used for finding the shortest path in a graph or a tree with a heuristic cost function. Here's an explanation of how IDA* works and its key components:

Components of IDA*:

1. **State Space Representation:**
 - IDA* operates on a state space where each state represents a configuration of the problem (e.g., a node in a graph, a position on a grid).
2. **Heuristic Function (h):**
 - A heuristic function estimates the cost from a given state to the goal state. In IDA*, this heuristic guides the search towards the goal efficiently. It must be admissible (never overestimates the true cost) for IDA* to find an optimal solution.
3. **Cost Function (g):**
 - The cost function $g(n)$ represents the actual cost from the start state to the current state n .
4. **Search Strategy:**
 - IDA* is an iterative deepening algorithm, meaning it performs a depth-first search with increasing depth limits until a solution is found. It combines depth-first search's memory efficiency with A*'s completeness and optimality.

Algorithm Steps:

1. **Initialization:**
 - Start with an initial state and set the initial depth limit bound to the estimated cost of the initial state (using the heuristic function).
2. **Iterative Deepening Loop:**
 - Perform a depth-first search with the current depth limit bound.
 - If a solution is found at depth d , return it.
 - If the depth limit is exceeded and no solution is found, increase the depth limit (set bound to the next minimum depth encountered during the failed search).
3. **Repeat:**
 - Repeat the search with the new depth limit until a solution is found.
4. **Termination:**
 - When a solution is found, it is guaranteed to be optimal due to the properties of the depth-first search with iterative deepening and the admissibility of the heuristic function.

Advantages and Disadvantages:

- **Advantages:**

- Memory efficient compared to traditional A* because it does not store all visited states.
- Guarantees finding an optimal solution when the heuristic function is admissible.
- Can handle large state spaces efficiently.
- **Disadvantages:**
 - Can be less efficient than A* in some cases due to repeated states at different depths.
 - The effectiveness depends heavily on the quality of the heuristic function.

Applications:

IDA* is commonly used in problems where:

- The state space is too large to store all visited states.
- Memory is a concern, and incremental deepening helps manage memory usage.
- Finding an optimal solution is crucial.

IDA* is a powerful algorithm for finding optimal paths in state spaces, combining the benefits of iterative deepening with A*'s heuristic guidance. Its efficiency and optimality make it a preferred choice in various domains requiring pathfinding or state space search.

24.What are heuristic functions? Explain the two heuristics used in 8 puzzle.

Also compute the same for the 8 puzzle instance given below :

A heuristic function or simply a heuristic is a function that ranks alternatives in various search algorithms at each branching step basing on an available information in order to make a decision which branch is to be followed during a search.

the two heuristics used in 8 puzzle are:

h_1 = the number of misplaced tiles. For Figure 4.7, all of the eight tiles are out of position, so the start state would have $h_1 = 8$. h_1 is an admissible heuristic, because it is clear that any tile that is out of place must be moved at least once.

h_2 = the sum of the distances of the tiles from their goal positions. Because tiles cannot move along diagonals, the distance we will count is the sum of the horizontal and vertical distances. This is sometimes called the city block distance or Manhattan distance. h_2 is also admissible, because all any move can do is move one tile one step

closer to the goal. Tiles 1 to 8 in the start state give a Manhattan distance of

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

25. Write a note on inventing heuristic functions.

Inventing admissible heuristic functions

To invent admissible heuristics we use Relaxed problems: A problem with fewer restrictions on the actions. The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem. The optimal solution to the original problem is, by definition also a solution to the relaxed problem.

the 8-puzzle actions are described formally as

A tile can move from square A to square B if

A is horizontally or vertically adjacent to B and B is blank

We can generate three relaxed problems by removing one or both of the conditions:

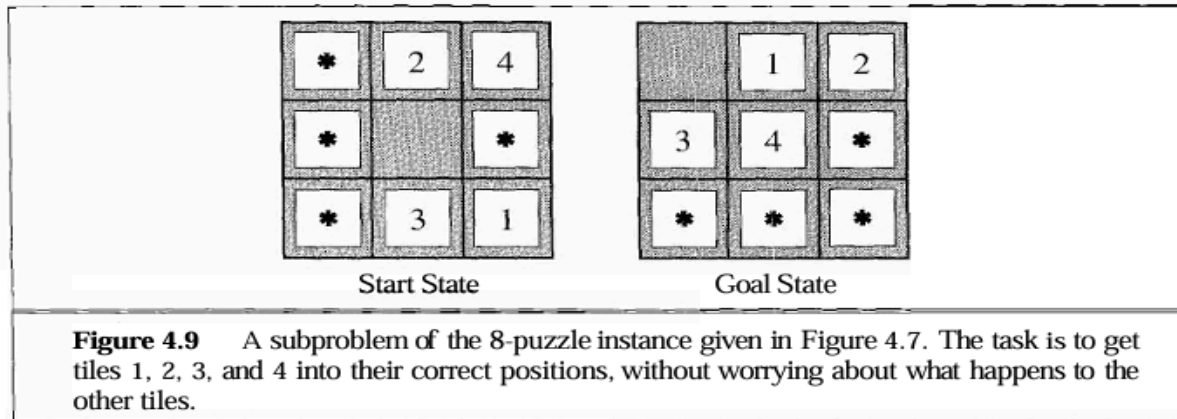
- (a) A tile can move from square A to square B if A is adjacent to B.
- (b) A tile can move from square A to square B if B is blank.
- (c) A tile can move from square A to square B.

From (a), we can derive h_2 (Manhattan distance) and from (c) we can derive h_1

A program called ABSOLVER can generate heuristics automatically from problem definitions, using the "relaxed problem" method and various other techniques. ABSOLVER generated a better new heuristic for the 8-puzzle and found the first useful heuristic for the famous Rubik's cube puzzle.

If a collection of admissible heuristics $h_1 \dots h_m$ is available for a problem, and none of them dominates any of the others, then $h(n) = \max \{h_1(n), \dots, h_m(n)\}$

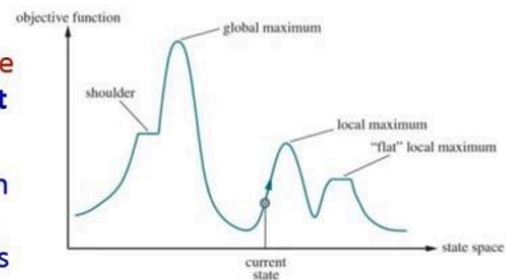
Admissible heuristics can also be derived from the solution cost of a subproblem of a given problem. For example, Figure below shows a subproblem of the 8-puzzle instance



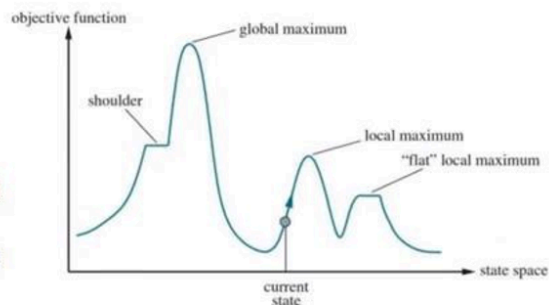
26. Explain Hill Climbing Search.

Hill-climbing Search Algorithm

- Hill climbing algorithm is a **Heuristic search** algorithm which continuously moves in the direction of **increasing value** to **find the peak of the mountain or best solution to the problem**.
- It keeps track of **one current state** and on each iteration moves to the **neighboring state with highest value**—that is, it heads in the direction that provides the **steepest ascent**.



- In this algorithm, when it reaches a peak value where no neighbor has a higher value, then it terminates.
- It is also called **greedy local search** as it only searches its good immediate neighbor state and not beyond that.
- Hill Climbing is mostly used when a good heuristic is available.



- Different regions in the state space landscape:
- **Local Maximum** is a state which is better than its neighbor states, but there is also another state which is higher than it. obj
- **Global Maximum** is the best possible state of state space landscape. It has the highest value of objective function.
- **Current state** is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum** is a flat space in the landscape where all the neighbor states of current states have the same value.
- **Shoulder** is a plateau region which has an uphill edge.

```

function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                   neighbor, a node

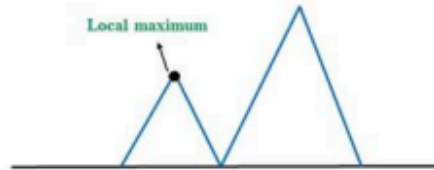
  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor

```

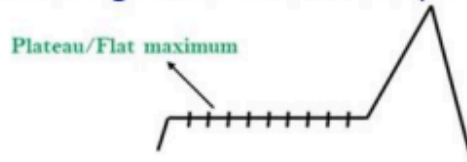
27. Explain the issues due to which hill climbing gets stuck.

Problems in Hill Climbing Algorithm

- **1. Local Maximum:** A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.
- **Solution:** Backtracking technique can be a solution of the local maximum in state space landscape.
- Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.



- **2. Plateau:** A plateau is the flat area of the search space in which all the neighbor states of the **current state** contains the **same value**, because of this algorithm does not find any best direction to move.
- A hill-climbing search might be lost in the plateau area.
- **Solution:** The solution for the plateau is to **take big steps** while searching, to solve the problem.
- **Randomly** select a state which is **far away from the current state** so it is possible that the algorithm could find non-plateau region.



28. Briefly Explain Simulated Annealing algorithm.

29. Compare various uninformed searching technique.

30. Explain genetic algorithm with example.

Genetic Algorithm

- A **genetic algorithm** (or **GA**) is a search technique used to find **true or approximate solutions**.
- Genetic algorithms are categorized as **global search heuristics**.
- GAs are particular class of evolutionary algorithms that use techniques inspired by **evolutionary biology** such as **inheritance, mutation, selection, and crossover** (also called recombination).

- The algorithm terminates when
- either a maximum number of generations has been produced,
- or a satisfactory fitness level has been reached for the population.

function GENETIC-ALGORITHM(*population*, *FITNESS-FN*) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new-population $\leftarrow$ empty set

loop for *i* **from** 1 **to** SIZE(*population*) **do**

x $\leftarrow$ RANDOM-SELECTION(*population*, *FITNESS-FN*)

y $\leftarrow$ RANDOM-SELECTION(*population*, *FITNESS-FN*)

child $\leftarrow$ REPRODUCE(*x*, *y*)

if (small random probability) **then** *child* $\leftarrow$ MUTATE(*child*)

add *child* to *new-population*

population $\leftarrow$ *new-population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to *FITNESS-FN*

function REPRODUCE(*x*, *y*) **returns** an individual

inputs: *x*, *y*, parent individuals

n $\leftarrow$ LENGTH(*x*)

c $\leftarrow$ random number from 1 to *n*

return APPEND(SUBSTRING(*x*, 1, *c*), SUBSTRING(*y*, *c* + 1, *n*))

Figure 4.17 A genetic algorithm. The algorithm is the same as the one diagrammed in Figure 4.15, with one variation: in this more popular version, each mating of two parents produces only one offspring, not two.

Steps in Genetic Algorithm

- Select Initial Population
- Fitness function
- Parent selection
- Crossover
- Mutation

Mixing Number

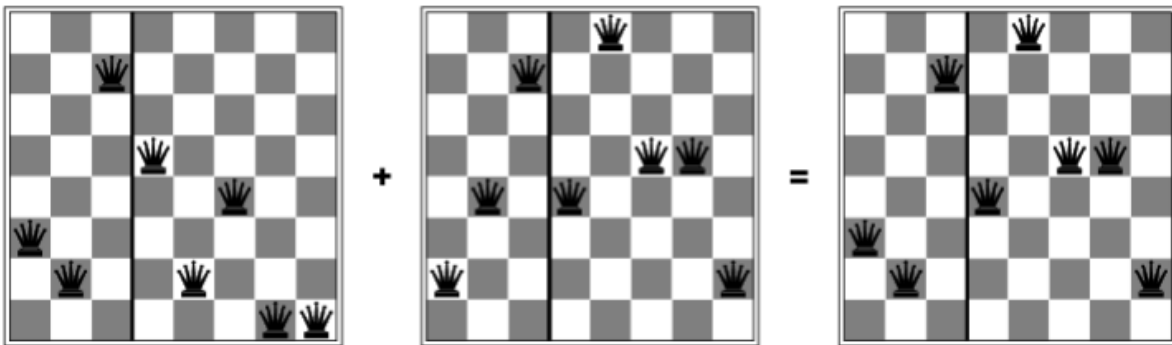
- The mixing number, p , which is the number of parents that come together to form offspring.
- The most common case is $p=2$: two parents combine their "genes"
- It is possible to have $p>2$, to simulate on computers.

Selection Process

- The **selection** process for selecting the individuals who will become the parents of the next generation:
- one possibility is to **select from all individuals** with probability proportional to their **fitness score**.
- Another possibility is to randomly select n individuals, ($n>p$) and then select p the **most fit ones** as parents.

The Recombination Procedure

- It is a random selection of **crossover point** to **split** each of the parent strings,
- and **recombine** the parts to form two children,
 - one with the first part of parent 1 and the second part of parent 2;
 - the other with the second part of parent 1 and the first part of parent 2.



31.Explain local beam search with example.

Keeping just one node in memory might seem to be an extreme reaction to the problem of memory limitations. The local beam search algorithm keeps track of k states rather than just one. It begins with k randomly generated states. At each step, all the successors of all k states are generated. If any one is a goal, the algorithm halts. Otherwise, it selects the k best successors from the complete list and repeats. At first sight, a local beam search with k states might seem to be nothing more than running k random restarts in parallel instead of in sequence. In fact, the two algorithms are quite different. In a random-restart search, each search process runs independently of the others. In a local beam search, useful information is

passed among the k parallel search threads. For example, if one state generates several good successors and the other $k - 1$ states all generate bad successors, then the effect is that the first state says to the others, "Come over here, the grass is greener!" The algorithm quickly abandons unfruitful searches and moves its resources to where the most progress is being made.

In its simplest form, local beam search can suffer from a lack of diversity among the k states—they can quickly become concentrated in a small region of the state space, making the search little more than an expensive version of hill climbing. A variant called stochastic beam search, analogous to stochastic hill climbing, helps to alleviate this problem. Instead of choosing the best k from the pool of candidate successors, stochastic beam search chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value. Stochastic beam search bears some resemblance to the process of natural selection, whereby the "successors" (offspring) of a "state" (organism) populate the next generation according to its "value" (fitness).

```

function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling the probability of downward steps

  current  $\leftarrow$  MAKE-NODE(INITIAL-STATE[problem])
  for  $t \leftarrow 1$  to  $\infty$  do
    T  $\leftarrow$  schedule[t]
    if T = 0 then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  VALUE[next] - VALUE[current]
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 

```
